

Summary

Cache Memory Systems: Architecture, Performance, and Mapping Techniques

1. Purpose and Performance Motivation

Cache memory is designed to increase system speed, not storage capacity. It acts as a high-speed buffer between the processor and main memory (DRAM), reducing average memory access time by exploiting predictable memory access patterns.

Key objective:
Bridge the speed gap between CPU and main memory using fast SRAM.

2. Memory Hierarchy Foundations

2.1 SRAM vs. DRAM

Feature	SRAM	DRAM
Speed	Very Fast	Slow
Storage Element	Flip-flops	Capacitors
Bit Transition	Single clock pulse	Requires charge/discharge
Cost	Expensive	Cheap
Usage	Cache Memory	Main Memory

SRAM is used for cache due to speed, while DRAM is used for main memory due to lower cost and larger capacity.

2.2 Principles of Locality

Cache efficiency relies on two access behaviors:

1. **Spatial Locality**

If a memory location is accessed, nearby locations are likely to be accessed soon.

→ Entire blocks are transferred instead of individual bytes.

2. **Temporal Locality**

If a location is accessed, it is likely to be accessed again in the near future.

2.3 Logical Memory Organization

- Main Memory → Divided into **Blocks**
- Cache Memory → Divided into **Lines**
- **Block size = Line size** (e.g., 32 or 64 bytes)
- Main memory has **many more blocks** than cache has lines

3. Cache Layouts and Access Methods

3.1 Look-Through vs. Look-Aside Cache

Feature	Look-Through (Hierarchical)	Look-Aside (Simultaneous)
Search Order	Cache → Main Memory	Cache and Main Memory together
Concurrency	High	Low
Bus Usage on Hit	Main memory bus free	Bus occupied
Miss Penalty	Cache time + Memory time	Only memory time
Best Use	Multi-processor systems	Uni-processor systems

4. Cache Consistency and Write Policies

Cache inconsistency occurs when updated data exists in cache but not in main memory.

4.1 Write Through

- Writes to cache and main memory simultaneously
- Guarantees 100% consistency
- No dirty bit required
- Slower due to DRAM write speed

- Acceptable because reads dominate (90% reads, 10% writes)

4.2 Write Back (Delayed Write)

- Writes only to cache
- Main memory updated later
- Requires Dirty Bit
- Faster writes
- Temporary inconsistency possible

4.3 Snooping

- Cache controller monitors other processors
- If another processor accesses a dirty block:
 - Cache updates main memory first
- Most efficient method
- Maintains consistency only when required

5. Cache Mapping Techniques

Mapping defines where a main memory block can be placed in cache.

5.1 Fully Associative Mapping

- Any block → Any cache line
- Requires searching all tags
- Largest tag size
- Best hit ratio
- High hardware cost

5.2 Direct Mapping (One-Way Set Associative)

Each block maps to exactly one cache line.

Formula:

Line Number = (Block Number) mod (Total Cache Lines)

- Single tag comparison
- Smallest tag size
- Fastest lookup
- Worst hit ratio due to conflict misses

5.3 Set-Associative Mapping (N-Way)

- Cache divided into sets
- Each set has N lines
- Block maps to one set but can occupy any line within it
- Only N tag comparisons required
- Reduces conflict misses

6. Address Splitting in Cache

Main memory addresses are divided into fields depending on mapping type.

Example Configuration:

- Main Memory: 4 GB
- Cache: 8 KB
- Block Size: 32 Bytes

Offset Calculation:

32 Bytes = 2^5 → Offset = 5 bits

Address Bit Breakdown

Mapping Type	Tag Bits	Index / Set Bits	Offset Bits	Total
Fully Associative	27	0	5	32
Direct Mapping	19	8	5	32
Two-Way Set Assoc.	20	7	5	32

- Offset → Block size dependent
- Index/Set → Cache structure dependent
- Tag → Remaining bits identifying memory block

7. Key Takeaways

- Cache improves performance, not capacity
- Effectiveness depends on locality of reference

- Mapping techniques represent a trade-off between:
 - Search speed
 - Hardware complexity
 - Hit ratio
- Direct mapping is simplest but least efficient
- N-way set associative mapping balances cost and performance
- High hit ratios ($\approx 95\%$ or more) are achievable due to locality

Final Insight

Cache design prioritizes logical efficiency over complexity.

A strong conceptual understanding of cache architecture and mapping techniques is essential for mastering Computer Organization and Architecture.